Отчет по лабораторной работе №3 предмета ООП

Выполнил Брусенцев Степан Эдуардович
Студент группы 6204-010302D

Задание на лабораторную работу:

Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Задание 1

Я ознакомился с документацией исключений входящих в API JAVA
java.lang.Exception
Базовый класс для всех проверяемых (checked) исключений. Обработка обязательна (try-catch или throws).

java.lang.IndexOutOfBoundsException
Непроверяемое исключение (unchecked). Выбрасывается при попытке доступа к индексу за пределами допустимого диапазона (например, в списке List).

java.lang.ArrayIndexOutOfBoundsException
Подкласс IndexOutOfBoundsException. Специально для массивов — обращение к несуществующему индексу массива.

java.lang.IllegalArgumentException
Непроверяемое исключение. Указывает на некорректный аргумент, переданный методу (например, отрицательный размер, null, где не допускается).

java.lang.IllegalStateException
Непроверяемое исключение. Возникает, когда метод вызывается в неподходящий момент (например, операция с закрытым потоком, вызов start() у уже запущенного потока).

Задание 2

В пакете functions создал два класса исключений:
FunctionPointIndexOutOfBoundsException
InappropriateFunctionPointException

```java
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException{  9 usages
    public FunctionPointIndexOutOfBoundsException(String message) { super(message); }
}
```

Рис.1 исключение FunctionPointIndexOutOfBoundsException

```java
package functions;

public class InappropriateFunctionPointException extends Exception{  21 usages
    public InappropriateFunctionPointException(String message) { super(message); }
}
```

Рис.2 исключение InappropriateFunctionPointException

Задание 3

Добавил в класс TabulatedFunction изменения реализующие исключения
В оба конструктора добавил исключение IllegalArgumentException, которое отвечает за корректное кол-во точек и задание границ определения.

```java
if (leftX>=rightX){
    throw new IllegalArgumentException("The left boundary of the domain must be less than the right one");
}
if (pointsCount<2){
    throw new IllegalArgumentException("The number of points must be more than 2");
}
```

```java
if (leftX>=rightX){
    throw new IllegalArgumentException("The left boundary of the domain must be less than the right one");
}
if (values.length<2){
    throw new IllegalArgumentException("The number of points must be more than 2");
}
```

Рис 3,4 конструкторы

В методы sgetPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY() и deletePoint() добавил выброс исключения FunctionPointIndexOutOfBoundsException, отвечающий за корректность индексов.

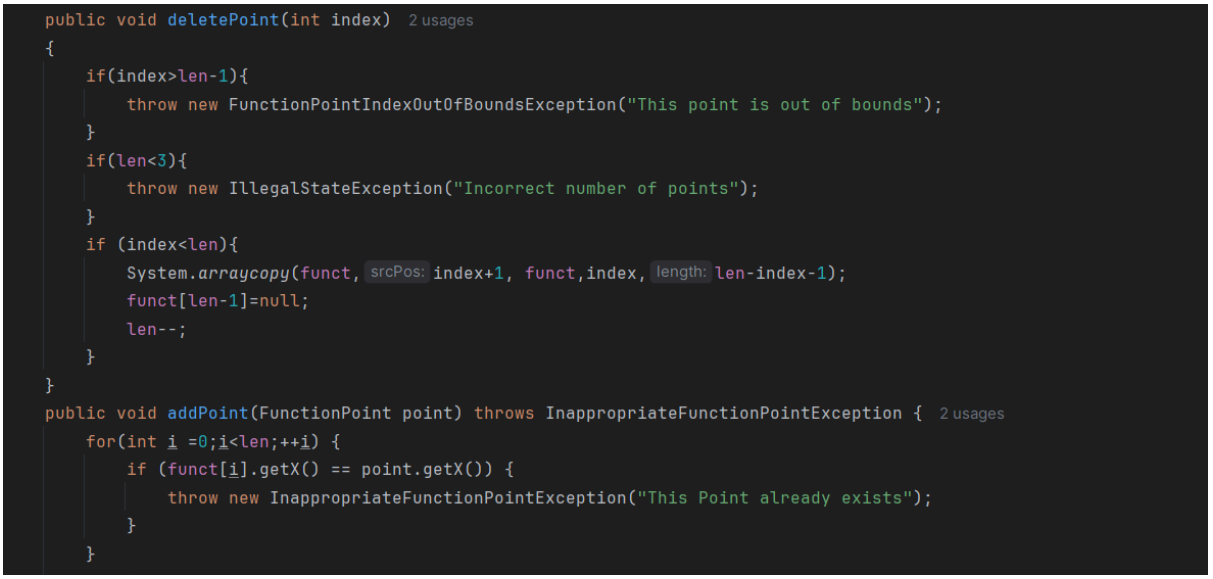
В методы setPoint() и setPointX() добавил выброс исключения InappropriateFunctionPointException, отвечающий за корректность координаты x.

В метод addPoint() также добавил это исключение, отвечающее за упорядоченность точек функции.

В метод deletePoint() добавил выброс исключения IllegalStateException обеспечивающий корректность кол-ва точек.

```
public FunctionPoint getPoint(int index)  1 usage
{
    if(index > len-1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    return new FunctionPoint(funct[index]);
}
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  1 usage
    if (index < len - 1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    if(index < len - 1 && funct[index+1].getX() > point.getX()){
        throw new InappropriateFunctionPointException("The point disrupts the order");
    }
    if(index > 0 && funct[index-1].getX() < point.getX()){
        throw new InappropriateFunctionPointException("The point disrupts the order");
    }
    funct[index]=new FunctionPoint(point);
}
```

```
public double getPointX(int index)  9 usages
{
    if(index>len-1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    return funct[index].getX();
}
public void setPointX(int index, double x) throws InappropriateFunctionPointException {  2 usages
    if (index < len - 1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    if(index < len - 1 && funct[index+1].getX() > x){
        throw new InappropriateFunctionPointException("The point disrupts the order");
    }
    if(index > 0 && funct[index-1].getX() < x){
        throw new InappropriateFunctionPointException("The point disrupts the order");
    }
    funct[index] = new FunctionPoint (x, funct[index].getY());
}
public double getPointY(int index)  4 usages
{
    if(index>len-1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    return funct[index].getY();
}
public void setPointY(int index, double y)  3 usages
{
    if(index>len-1){
        throw new FunctionPointIndexOutOfBoundsException("Going beyond the set of points");
    }
    funct[index] = new FunctionPoint (funct[index].getX(),y);
}
```



```
public void deletePoint(int index)  2 usages
{
    if(index>len-1){
        throw new FunctionPointIndexOutOfBoundsException("This point is out of bounds");
    }
    if(len<3){
        throw new IllegalStateException("Incorrect number of points");
    }
    if (index<len){
        System.arraycopy(funct, srcPos: index+1, funct,index, length: len-index-1);
        funct[len-1]=null;
        len--;
    }
}
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  2 usages
    for(int i =0;i<len;++i) {
        if (funct[i].getX() == point.getX()) {
            throw new InappropriateFunctionPointException("This Point already exists");
        }
    }
```

Рис 5,6,7 реализация исключений в методах

Задание 4

Создал LinkedListTabulatedFunction класс, в котором реализован двусвязный циклический список
Описал класс элементов списка FunctionNode, содержащий информационное поле для хранения данных типа FunctionPoint.

Описал класс LinkedListTabulatedFunction объектов списка, содержащий поле ссылки на объект головы, а также иные вспомогательные поля.

Реализовал метод FunctionNode getNodeByIndex(int index), возвращающий ссылку на объект элемента списка по его номеру.

Реализовал метод FunctionNode addNodeToTail(), добавляющий новый элемент в конец списка и возвращающий ссылку на объект этого элемента.

Реализовал метод FunctionNode addNodeByIndex(int index), добавляющий новый элемент в указанную позицию списка и возвращающий ссылку на объект этого элемента.

Реализовал метод FunctionNode deleteNodeByIndex(int index), удаляющий элемент списка по номеру и возвращающий ссылку на объект удаленного элемента.

```java
private static class FunctionNode {  40 usages
    private FunctionPoint value;  24 usages
    private FunctionNode next;  18 usages
    private FunctionNode prev;  17 usages

    FunctionNode(FunctionPoint value) { this.value = value == null ? null : new FunctionPoint(value); }
}
```

```java
// Голова списка (не хранит данных, всегда присутствует)
private final FunctionNode head;  27 usages
// Текущее количество значащих элементов (без головы)
private int size;  17 usages

// Кэш последнего обращения для оптимизации getNodeByIndex
private FunctionNode cachedNode;  10 usages
private int cachedIndex;  19 usages

public LinkedListTabulatedFunction() {  no usages
    head = new FunctionNode( value: null);
    head.next = head;
    head.prev = head;
    size = 0;
    cachedNode = null;
    cachedIndex = -1;
}
```

```java
FunctionNode getNodeByIndex(int index) {  9 usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }

    // Выбор стартовой точки обхода: голова слева, голова справа или кэш
    FunctionNode startNode;
    int startIndex;

    boolean cacheValid = cachedNode != null && cachedIndex >= 0 && cachedIndex < size;
    int fromCache = cacheValid ? Math.abs(index - cachedIndex) : Integer.MAX_VALUE;
    int fromHead = index;              // расстояние от head.next
    int fromTail = size - 1 - index;   // расстояние от head.prev

    if (fromCache <= fromHead && fromCache <= fromTail) {
        startNode = cachedNode;
        startIndex = cachedIndex;
    } else if (fromHead <= fromTail) {
        startNode = head.next;
        startIndex = 0;
    } else {
        startNode = head.prev;
        startIndex = size - 1;
    }

    FunctionNode current = startNode;
    if (startIndex < index) {
        for (int i = startIndex; i < index; i++) {
            current = current.next;
        }
    } else if (startIndex > index) {
        for (int i = startIndex; i > index; i--) {
            current = current.prev;
        }
    }
```

```
        cachedNode = current;
        cachedIndex = index;
        return current;
    }
```

```
    FunctionNode addNodeToTail() {  1 usage
        FunctionNode node = new FunctionNode( value: null);
        insertBefore(head, node);
        return node;
    }
```

```
FunctionNode addNodeByIndex(int index) {  no usages
    if (index < 0 || index > size) {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }
    if (index == size) {
        return addNodeToTail();
    }
    FunctionNode target = getNodeByIndex(index);
    FunctionNode node = new FunctionNode( value: null);
    insertBefore(target, node);
    // Сдвигаем кэш, если вставили раньше него
    if (cachedIndex >= index) {
        cachedIndex++;
    }
    return node;
}
```

```
FunctionNode deleteNodeByIndex(int index) {  no usages
    FunctionNode node = getNodeByIndex(index);
    unlink(node);
    if (cachedNode == node) {
        cachedNode = null;
        cachedIndex = -1;
    } else if (cachedIndex > index) {
        cachedIndex--;
    }
    return node;
}
```

Рис.8-12 методы реализованные в соответствии с заданием 4

Задание 5

Реализовал конструкторы и методы аналогичные методам и конструкторам класса TabulatedFunction.

Задание 6

Класс TabulatedFunction переименовал в класс ArrayTabulatedFunction. Создал интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction. И сделал чтобы оба класса реализовывали этот интерфейс

```
package functions;

public interface TabulatedFunction {  7 usages  2 implementations
    double getLeftDomainBorder();  6 usages  2 implementations
    double getRightDomainBorder();  6 usages  2 implementations
    double getFunctionValue(double x);  6 usages  2 implementations
    int getPointsCount();  6 usages  2 implementations
    void getPoint(int index);  1 usage  2 implementations
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;  1 usage  2 implementations
    double getPointX(int index);  9 usages  2 implementations
    void setPointX(int index, double x) throws InappropriateFunctionPointException;  2 usages  2 implementations
    double getPointY(int index);  4 usages  2 implementations
    void setPointY(int index, double y);  3 usages  2 implementations
    void deletePoint(int index);  2 usages  2 implementations
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  2 usages  2 implementations
}
```

```
public class LinkedListTabulatedFunction implements TabulatedFunction {  2 usages
```

```
public class ArrayTabulatedFunction implements TabulatedFunction {  5 usages
```

Рис. 13-15 Реализация интерфейса

Задание 7

Проверил работу написанных классов.

```
1. СОЗДАНИЕ КВАДРАТИЧНОЙ ФУНКЦИИ f(x) = x² - 3x + 2:
Исходная функция:
Индекс  X       Y
----------------------------
0   0,0000  2,0000
1   1,0000  0,0000
2   2,0000  0,0000
3   3,0000  2,0000
4   4,0000  6,0000

2. ВЫЧИСЛЕНИЕ ЗНАЧЕНИЙ В РАЗЛИЧНЫХ ТОЧКАХ:
x=-1,0: вне области определения [0,0, 4,0]
x=0,0: интерполированное=2,0000, точное=2,0000 (погрешность=0,0000)
x=0,5: интерполированное=1,0000, точное=0,7500 (погрешность=0,2500)
x=1,0: интерполированное=0,0000, точное=0,0000 (погрешность=0,0000)
x=1,5: интерполированное=0,0000, точное=-0,2500 (погрешность=0,2500)
x=2,0: интерполированное=0,0000, точное=0,0000 (погрешность=0,0000)
x=2,5: интерполированное=1,0000, точное=0,7500 (погрешность=0,2500)
x=3,0: интерполированное=2,0000, точное=2,0000 (погрешность=0,0000)
x=3,5: интерполированное=4,0000, точное=3,7500 (погрешность=0,2500)
x=4,0: интерполированное=6,0000, точное=6,0000 (погрешность=0,0000)
x=5,0: вне области определения [0,0, 4,0]
```

```
3. ДОБАВЛЕНИЕ ТОЧКИ (2.5, 0.75):
Количество точек после добавления: 6
После добавления точки:
Индекс  X       Y
----------------------------
0   0,0000  2,0000
1   1,0000  0,0000
2   2,0000  0,0000
3   2,5000  0,7500
4   3,0000  2,0000
5   4,0000  6,0000

4. ИЗМЕНЕНИЕ ТОЧКИ:
Попытка изменить X точки 3 на 3.2:
До: (2,50, 0,75)
Нарушен порядок точек по x Going beyond the set of points
После: (2,50, 0,75)
Изменение Y точки 3 на 10.0:
После: (2,50, 10,00)
```

```
5. УДАЛЕНИЕ ТОЧКИ 2:
Количество точек после удаления: 5
После удаления точки:
Индекс  X       Y
----------------------------
0   0,0000  2,0000
1   1,0000  0,0000
2   2,5000  10,0000
3   3,0000  2,0000
4   4,0000  6,0000

6. ПРОВЕРКА ИНТЕРПОЛЯЦИИ ПОСЛЕ МОДИФИКАЦИЙ:
f(1,2) = 1,3333 (точное=-0,1600, отклонение=1,4933)
f(2,0) = 6,6667 (точное=0,0000, отклонение=6,6667)
f(2,8) = 5,2000 (точное=1,4400, отклонение=3,7600)
f(3,5) = 4,0000 (точное=3,7500, отклонение=0,2500)

7. ГРАНИЧНЫЕ ЗНАЧЕНИЯ:
Область определения: [0,00, 4,00]
f(0,00) = 2,0000
f(4,00) = 6,0000
```

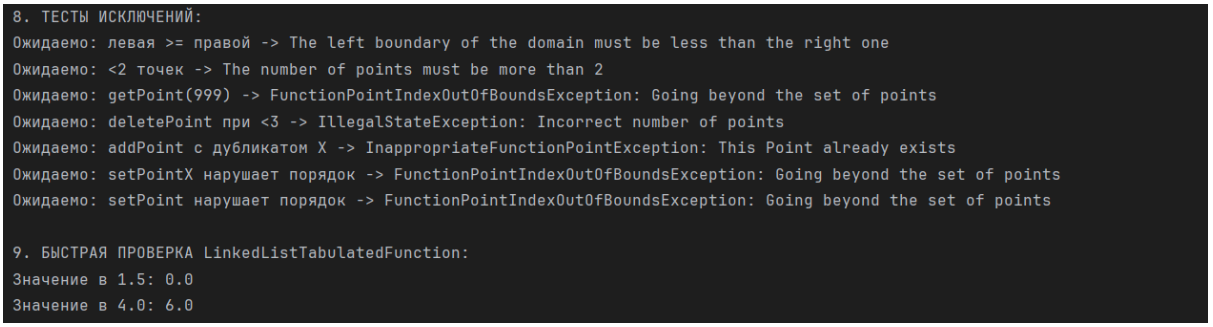


Рис. 15-19 Проверка работы программы

Все задания лабораторной работы №3 выполнены